

 Marwadi University Marwadi Chaudhara Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Simulate linear convolution and circular convolution on discrete time signals.	
Experiment No: 2	Date: 25-07-2026	Enrollment No: 92400133037

Theory:

Linear convolution and circular convolution are mathematical operations used to combine two signals to obtain a third signal. They are widely used in various applications, such as signal processing, image processing, and audio processing.

Linear convolution calculates the sum of element-wise products of two signals, considering the full range of valid indices. It is typically used for finite-length signals and can produce an output signal that is longer than the input signals.

Circular convolution, on the other hand, calculates the sum of element-wise products of two signals, considering a periodic extension of the input signals. It is commonly used for periodic or infinite-length signals and produces an output signal with the same length as the input signals.

Flowchart:

The flowchart for the program will consist of the following steps:

Define two discrete-time signals (signal 1 and signal 2).

Compute the linear convolution of the two signals using `numpy.convolve`.

Compute the circular convolution of the two signals using `numpy.fft.ifft`.

Plot and display the linear and circular convolution results.

Save the linear and circular convolution results (optional).

Now, let's see the Python program that simulates linear convolution and circular convolution on discrete-time signal

Program:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
def linear_convolution(signal1, signal2):
```

```
    # Compute the linear convolution
```

```
    linear_conv = np.convolve(signal1, signal2, mode='full')
```

```
    return linear_conv
```

```
def circular_convolution(signal1, signal2):
```

```
    # Compute the circular convolution
```

```
    # Compute the circular convolution
```

```
    If( len(signal1) > len(signal2) ):
```

```
        fft_length=len(signal1);
```

```
    else:
```

```
        fft_length=len(signal2);
```

```
    fft_signal1 = np.fft.fft(signal1, fft_length)
```

```
    fft_signal2 = np.fft.fft(signal2, fft_length)
```

```
    circular_conv = np.fft.ifft(fft_signal1 * fft_signal2)
```

```
    return circular_conv
```

 Marwadi University Marwadi Chaudhara Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Simulate linear convolution and circular convolution on discrete time signals.	
Experiment No: 2	Date: 25-07-2026	Enrollment No: 92400133037

```

# Define the discrete-time signals
signal1 = np.array([1, 2, 3, 4, 5])
signal2 = np.array([2, 4, 6, 8, 10])

# Compute the linear convolution
linear_conv = linear_convolution(signal1, signal2)

# Compute the circular convolution
circular_conv = circular_convolution(signal1, signal2)

# Plot the linear and circular convolution results
plt.figure(figsize=(10, 6))
plt.subplot(2, 1, 1)
plt.stem(linear_conv)
plt.title('Linear Convolution')
plt.xlabel('Sample')
plt.ylabel('Amplitude')

plt.subplot(2, 1, 2)
plt.stem(circular_conv)
plt.title('Circular Convolution')
plt.xlabel('Sample')
plt.ylabel('Amplitude')

plt.tight_layout()
plt.show()

# Save the linear and circular convolution results (optional)
# np.savetxt('linear_convolution.txt', linear_conv, delimiter=',')
# np.savetxt('circular_convolution.txt', circular_conv, delimiter=',')

```

Explanation:

We import the required modules, including numpy for array operations and matplotlib.pyplot for plotting.

The linear_convolution function computes the linear convolution between two signals using numpy.convolve with the mode set to 'full'.

The circular_convolution function computes the circular convolution between two signals using numpy.fft.fft and numpy.fft.ifft. It first calculates the FFT of both signals with a specified length and then computes the inverse FFT of the element-wise product.

In the main part of the program, we define two discrete-time signals (signal1 and signal2).

We compute the linear convolution by calling the linear_convolution function with signal1 and signal2.

 Marwadi University Marwadi Chhindvan Group 	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Simulate linear convolution and circular convolution on discrete time signals.	
Experiment No: 2	Date: 25-07-2026	Enrollment No: 92400133037

We compute the circular convolution by calling the `circular_convolution` function with `signal1` and `signal2`.

We plot and display the linear and circular convolution results using `matplotlib.pyplot.stem`.

The linear and circular convolution results can be optionally saved to files using `numpy.savetxt`.

You can modify the `signal1` and `signal2` variables to use different discrete-time signals for convolution.

Uncomment the `np.savetxt` lines if you want to save the linear and circular convolution results to text files.

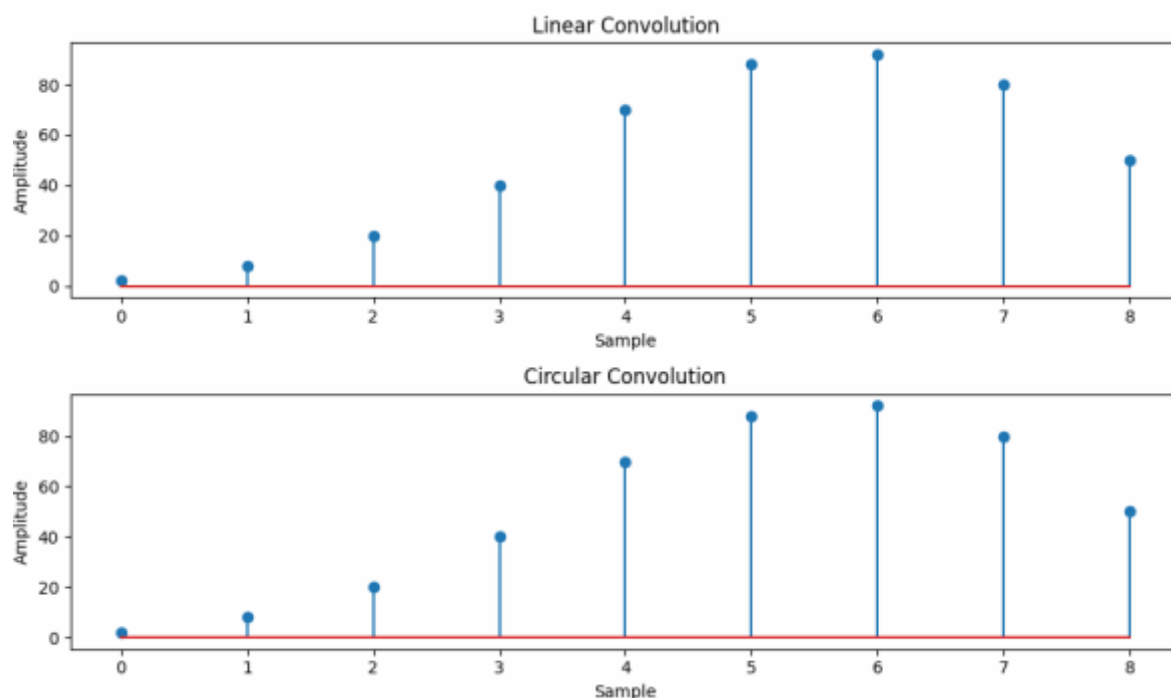
Make sure you have `numpy` and `matplotlib` installed (`pip install numpy matplotlib`) before running this program.

Expected Output:

A plot will be displayed showing the linear and circular convolution results.

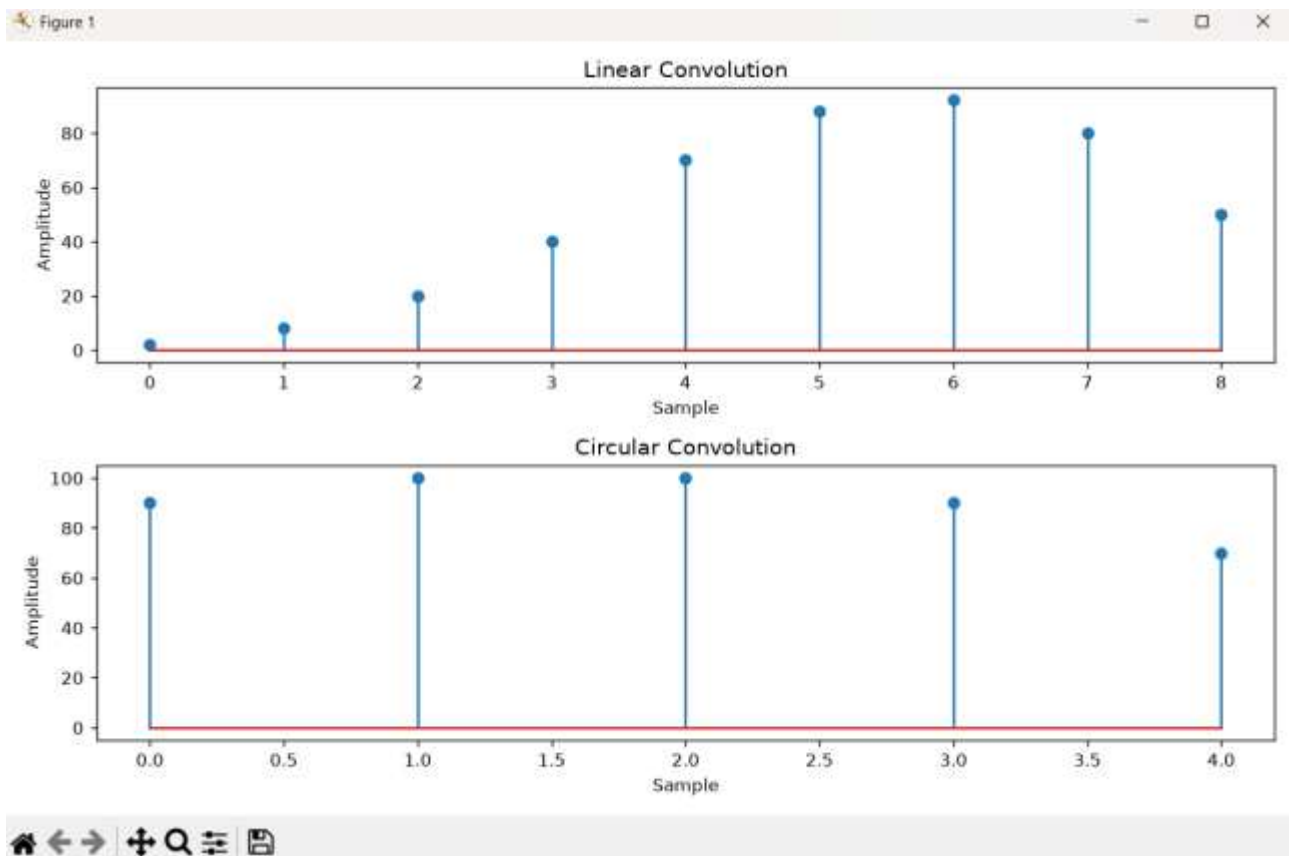
The linear and circular convolution results will be saved to text files (optional).

Feel free to experiment with different discrete-time signals and observe the linear and circular convolution outputs.



 Marwadi University Marwadi Chhindvan Group	Marwadi University Faculty of Engineering & Technology Department of Information and Communication Technology	
Subject: Digital Signal and Image Processing (01CT1513)	AIM: Simulate linear convolution and circular convolution on discrete time signals.	
Experiment No: 2	Date: 25-07-2026	Enrollment No: 92400133037

Result / Output:



Conclusion:

The experiment was successfully carried out to simulate both linear convolution and circular convolution of two discrete-time signals using Python. The results obtained from the program clearly demonstrate the difference between the two convolution techniques. Linear convolution produces an output whose length is greater than the input signals because it considers all possible overlaps between them, whereas circular convolution assumes the signals are periodic and produces an output with a length equal to the FFT size. The graphical representation made it easy to compare the two results and understand their behavior. Overall, this experiment helped in understanding the practical implementation of convolution operations using NumPy and reinforced the importance of choosing the appropriate convolution method depending on the application in digital signal and image processing.